

DOCUMENTACION TECNICA PRUEBA TECNICA

Nombre: Carlos Alejandro Maldonado López

Cargo: Desarrollador de Software

Prueba SQL

The screenshot displays three SQL queries in SQL Server Enterprise Manager, each preceded by a comment. The first query lists all products with their manufacturer ID, product ID, description, price, and price including 10% VAT. The second query shows the total stock quantity for each product description. The third query shows the average price for each manufacturer. Below the queries, the 'Results' pane shows the output of the first query as a table with 8 rows.

```
--* Obtenga una lista de todos los productos utilizando sentencias SQL, indicando para cada uno su id_fabricante,
--SELECT
    id_fabricante,
    id_producto,
    descripcion,
    precio,
    ROUND(precio * 1.10, 2) AS precio_con_iva
FROM productos;

--* Obtenga la cantidad total de existencias por producto.
--SELECT
    descripcion,
    SUM(existencias) AS total_existencias
FROM productos
GROUP BY descripcion;

--* Obtenga el promedio de precio por fabricante.
--SELECT
    id_fabricante,
    ROUND(AVG(precio), 2) AS promedio precio
```

	id_fabricante	id_producto	descripcion	precio	precio_con_iva
1	Aci	41001	Aguja	58.00	63.8000
2	Aci	41002	Micropore	80.00	88.0000
3	Aci	41003	Gasa	112.00	123.2000
4	Aci	41004	Equipo macrogoteo	110.00	121.0000
5	Bic	41003	Curas	120.00	132.0000
6	Bic	Xk47	Compresa	200.00	220.0000
7	Inc	41089	Canaleta	500.00	550.0000
8	Qsa	Xk47	Compresa	150.00	165.0000

--* Obtenga la cantidad total de existencias por producto.

```
--SELECT
    descripcion,
    SUM(existencias) AS total_existencias
FROM productos
GROUP BY descripcion;
```

--* Obtenga el promedio de precio por fabricante.

```
--SELECT
    id fabricante,
    ROUND(AVG(precio),2) AS promedio_precio
FROM productos
GROUP BY id fabricante;
```

--* Obtenga el producto con mayor precio.

```
--SELECT
    TOP 1 id fabricante,
    id producto,
    descripcion,
    precio
```

75 %

Results Messages

6	Bic	Xk4/	Compresa	200.00	220.0000
7	Inc	41089	Canaleta	500.00	550.0000
8	Qsa	Xk47	Compresa	150.00	165.0000

	descripcion	total_existencias
1	Aguja	227
2	Canaleta	30
3	Compresa	400
4	Curas	20
5	Equipo m...	50
6	Gasas	80
7	Micropore	150

```

--* Obtenga el producto con mayor precio.
SELECT
TOP 1 id_fabricante,
      id_producto,
      descripcion,
      precio
FROM productos
ORDER BY precio DESC
;

--* Cargar en el sistema un nuevo pedido de 500 Curas que envió el fabricante Bic.
UPDATE productos
SET existencias = existencias + 500
WHERE id_fabricante = 'Bic' AND descripcion = 'Curas';

--* El fabricante Osa ya no es nuestro proveedor y se deben eliminar sus productos de la BD.
DELETE FROM productos

```

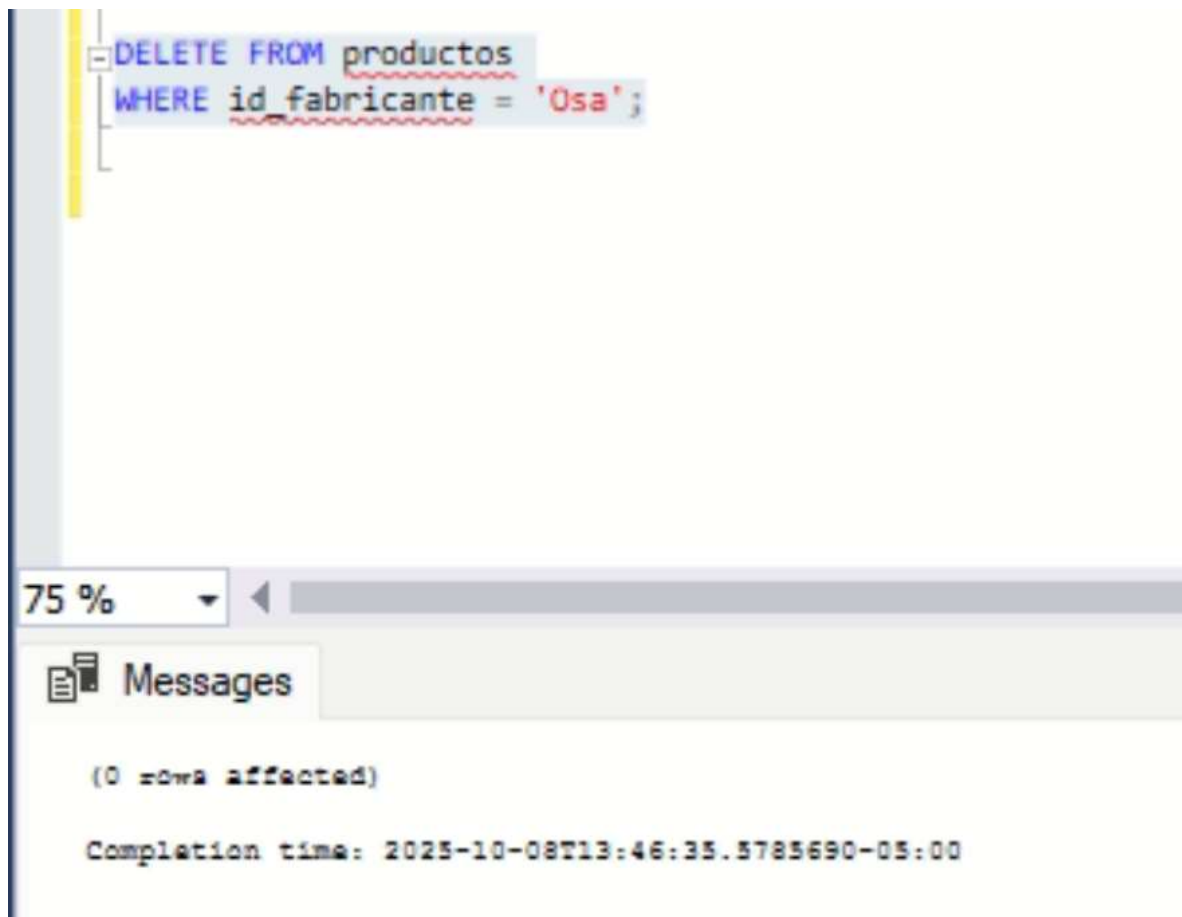
75 %

Results Messages

4	Curas	20
5	Equipo m...	50
6	Gasa	80
7	Micropore	150

	id_fabricante	promedio_precio
1	Aci	90.000000
2	Bic	160.000000
3	Inc	500.000000
4	Qsa	150.000000

	id_fabricante	id_producto	descripcion	precio
1	Inc	41089	Canaleta	500.00



A continuación anexo el SCRIPT

--• Obtenga una lista de todos los productos utilizando sentencias SQL, indicando para cada uno su id_fabricante, id_producto, descripción, precio y precio con I.V.A. incluido (es el precio anterior aumentado en un 10%).

SELECT

id_fabricante,

id_producto,

descripcion,

precio,

ROUND(precio * 1.10, 2) AS precio_con_iva

FROM productos;

--• Obtenga la cantidad total de existencias por producto.

```
SELECT
    descripcion,
    SUM(existencias) AS total_existencias
FROM productos
GROUP BY descripcion;
```

--• Obtenga el promedio de precio por fabricante.

```
SELECT
    id_fabricante,
    ROUND(AVG(precio),2) AS promedio_precio
FROM productos
GROUP BY id_fabricante;
```

--• Obtenga el producto con mayor precio.

```
SELECT
TOP 1 id_fabricante,
    id_producto,
    descripcion,
    precio
FROM productos
ORDER BY precio DESC
```

;

--• Cargar en el sistema un nuevo pedido de 500 Curas que envió el fabricante Bic.

UPDATE productos

SET existencias = existencias + 500

WHERE id_fabricante = 'Bic' AND descripcion = 'Curas';

--• El fabricante Osa ya no es nuestro proveedor y se deben eliminar sus productos de la BD.

DELETE FROM productos

WHERE id_fabricante = 'Osa';

Paso a paso

Generar contenedor DOCKER para lograr distribuir eficaz y de manera rápida el proyecto así como replicarlo y generar ambientes rápidos con imágenes de tipo Linux

Como yo ya tengo configurado mi ambiente de WSL y docker solo realizo la creación de mis archivos DOCKERFILE y docker-compose para crear la imagen.

- Requerimientos

- Base de datos MYSQL
- Base de datos SQL independientes para ejercicios SQL
- PHP usaremos la versión PHP 8.2
- Composer – Lo usaremos para utilizar componentes como paquetes además de indexar los archivos y proyecto
- Laravel – Lo usaremos como framework al igual que Bootstrap

Logramos crear la imagen docker y en ella ya empezamos a crear nuestro proyecto de laravel.

```
1 FROM php:8.2-apache-bullseye
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 RUN apt-get update && apt-get install -y \
6     git unzip curl libzip-dev libpng-dev libjpeg62-turbo-dev libfreetype6-dev \
7     && docker-php-ext-configure gd --with-zlib --with-freetype \
8     && docker-php-ext-install pdo pdo_mysql gd zip opcache \
9     && a2enmod rewrite \
10    && rm -rf /var/lib/apt/lists/*
11
12 RUN sed -i 's|/var/www/html|/var/www/html/laravel/public|g' /etc/apache2/sites-enabled/000-default.conf \
13    && echo "<Directory /var/www/html/laravel/public>\n    AllowOverride All\n    Require all granted\n</Directory>" >> /etc/apache2/apache2.conf
14
15 RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer
16
17 WORKDIR /var/www/html
18
19 EXPOSE 80
20
21 CMD ["apache2-foreground"]
22
```

```

1 services:
2   app:
3     build: .
4     container_name: laravel_app
5     ports:
6       - "8080:80"
7     volumes:
8       - ../laravel:/var/www/html/laravel
9     environment:
10      APP_ENV: local
11      DB_CONNECTION: mysql
12      DB_HOST: db
13      DB_PORT: 3306
14      DB_DATABASE: laravel
15      DB_USERNAME: laravel
16      DB_PASSWORD: laravelpass
17     depends_on:
18       - db
19
20   db:
21     image: mysql:8.0
22     container_name: laravel_db
23     command: --default-authentication-plugin=mysql_native_password
24     restart: always
25     environment:
26       MYSQL_ROOT_PASSWORD: rootpass
27       MYSQL_DATABASE: laravel
28       MYSQL_USER: laravel
29       MYSQL_PASSWORD: laravelpass
30     ports:
31       - "3306:3306"
32     volumes:
33       - db_data:/var/lib/mysql
34
35   phpmyadmin:
36     image: phpmyadmin:5
37     container_name: laravel_phpmyadmin
38     environment:
39       PMA_HOST: db
40       PMA_USER: laravel
41       PMA_PASSWORD: laravelpass
42       UPLOAD_LIMIT: 256M
43     ports:
44       - "8881:80"
45     depends_on:
46       - db
47
48   volumes:
49     db_data:
50

```

Luego de ya haber creado nuestro proyecto donde se almacenara la carpeta laravel y allí a su vez nuestra aplicación y de igual manera se conectara a nuestra base de datos laravel, comprobamos y garantizamos el servicio de todo.



Luego de ello ya iniciamos con la creación del proyecto en laravel ingresando los comandos de composer así generando nuestro contenido y de igual manera haciendo un migrate para que se creen las tablas principales del proyecto.

Ahora realizaremos la creación de las tablas que nos solicitan en un complemento que añadimos al composer que se llama PHPMYADMIN para poder visualizar la base de datos desde allí formando una pila LAMP.

Creamos los objetos de las tablas en laravel

```
root@c2879293d0e5:/var/www/html/laravel# php artisan make:migration create_municipios_table

INFO Migration [database/migrations/2025_10_08_132113_create_municipios_table.php] created successfully.

root@c2879293d0e5:/var/www/html/laravel# php artisan make:migration create_tipos_documento_table

INFO Migration [database/migrations/2025_10_08_132122_create_tipos_documento_table.php] created successfully.

root@c2879293d0e5:/var/www/html/laravel# php artisan make:migration create_genero_table

INFO Migration [database/migrations/2025_10_08_132133_create_genero_table.php] created successfully.

root@c2879293d0e5:/var/www/html/laravel# php artisan make:migration create_paciente_table

INFO Migration [database/migrations/2025_10_08_132142_create_paciente_table.php] created successfully.
```

Modificamos los schemas para adaptarlo con nuestra estructura

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('paciente', function (Blueprint $table) {
            $table->id();
            $table->foreignId('tipo_documento_id')->constrained('tipos_documento')->onDelete('restrict');
            $table->string('numero_documento', 20);
            $table->string('nombre1', 100);
            $table->string('nombre2', 100)->nullable();
            $table->string('apellido1', 100);
            $table->string('apellido2', 100)->nullable();
            $table->foreignId('genero_id')->constrained('genero')->onDelete('restrict');
            $table->foreignId('departamento_id')->constrained('departamentos')->onDelete('restrict');
            $table->foreignId('municipio_id')->constrained('municipios')->onDelete('restrict');
            $table->string('correo', 150);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('paciente');
    }
};
```

Generamos

los

seeders

```
INFO Seeder [database/seeders/MunicipiosSeeder.php] created successfully.

root@c2879293d0e5:/var/www/html/laravel# php artisan make:seeder TiposDocumentoSeeder

INFO Seeder [database/seeders/TiposDocumentoSeeder.php] created successfully.

root@c2879293d0e5:/var/www/html/laravel# php artisan make:seeder GeneroSeeder

INFO Seeder [database/seeders/GeneroSeeder.php] created successfully.

root@c2879293d0e5:/var/www/html/laravel# php artisan make:seeder PacienteSeeder

INFO Seeder [database/seeders/PacienteSeeder.php] created successfully.

root@c2879293d0e5:/var/www/html/laravel# php artisan make:seeder UsersSeeder

INFO Seeder [database/seeders/UsersSeeder.php] created successfully.
```

Agregamos

cada

seeder

y

los

datos

principales

```
namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class DepartamentosSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        //
        DB::table('departamentos')->insert([
            ['nombre' => 'Antioquia'],
            ['nombre' => 'Cundinamarca'],
            ['nombre' => 'Valle del Cauca'],
            ['nombre' => 'Bolívar'],
            ['nombre' => 'Santander'],
        ]);
    }
}
```

para que los contenga , agregamos el facade de DB.

Para el caso de usuario agregamos el facade de hash para contener seguridad en los accesos

```
database / seeders / UsersSeeder.php / UsersSeeder / Run
<?php

namespace Database\Seeders;

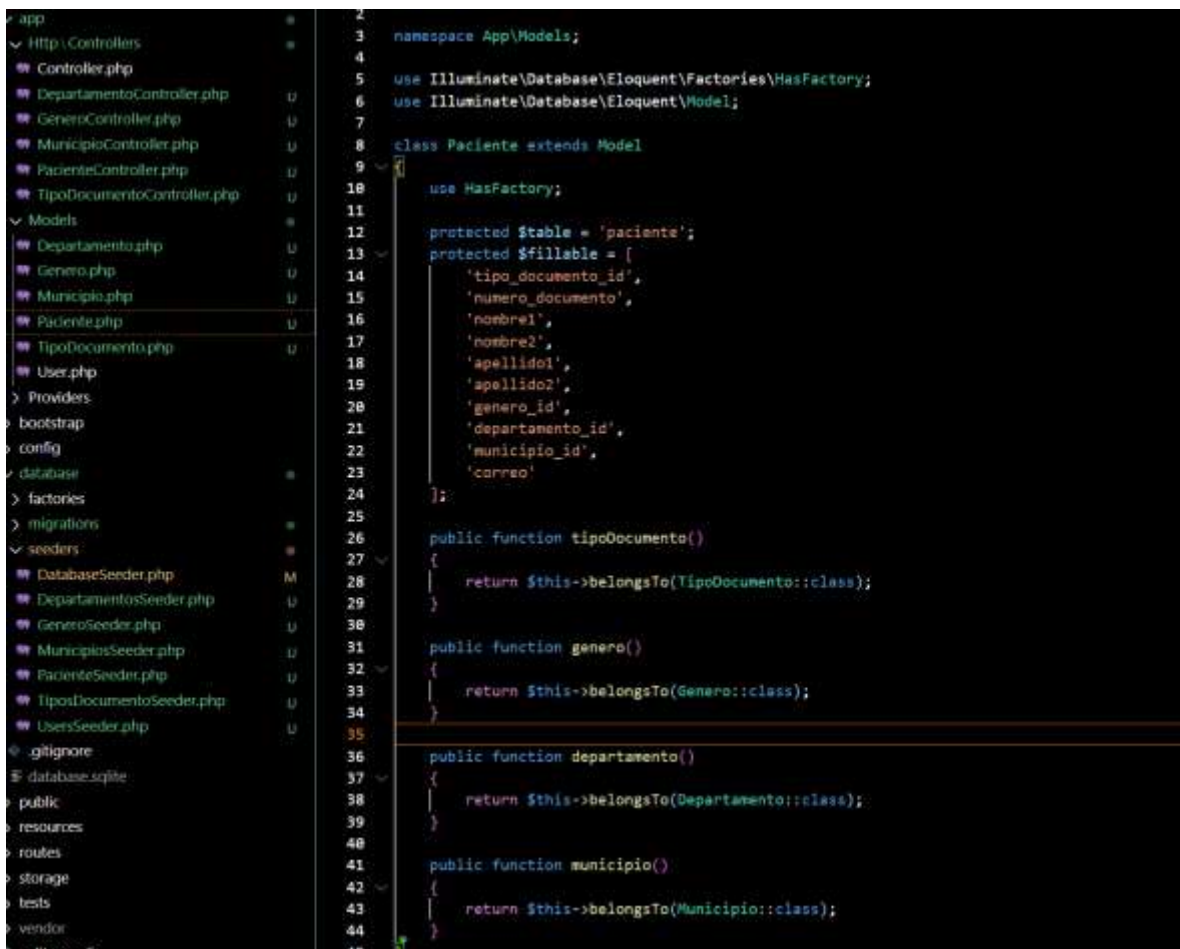
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\DB;

class UsersSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        //
        DB::table('users')->insert([
            'name' => 'Administrador',
            'email' => 'admin@sinergia.com',
            'password' => Hash::make('123Sinergia.'),
            'created_at' => now(),
        ]);
    }
}
```

Ya teniendo nuestros pacientes y ejecutando los seeders , ya con la data en la base de datos, iniciamos a realizar la creación de los modelos.

```
INFO Controller [app/Http/Controllers/TipoDocumentoController.php] created successfully.
root@c2879293d0e5:/var/www/html/laravel# php artisan make:model Genero -mcr
INFO Model [app/Models/Genero.php] created successfully.
INFO Migration [database/migrations/2025_10_08_133939_create_generos_table.php] created successfully.
INFO Controller [app/Http/Controllers/GeneroController.php] created successfully.
root@c2879293d0e5:/var/www/html/laravel# php artisan make:model Paciente -mcr
INFO Model [app/Models/Paciente.php] created successfully.
INFO Migration [database/migrations/2025_10_08_133949_create_pacientes_table.php] created successfully.
INFO Controller [app/Http/Controllers/PacienteController.php] created successfully.
```

Tendríamos nuestro modelo y nuestro controlador.



```
1 namespace App;
2 namespace App\Models;
3
4 use Illuminate\Database\Eloquent\Factories\HasFactory;
5 use Illuminate\Database\Eloquent\Model;
6
7 class Paciente extends Model
8 {
9     use HasFactory;
10
11     protected $table = 'paciente';
12     protected $fillable = [
13         'tipo_documento_id',
14         'numero_documento',
15         'nombre1',
16         'nombre2',
17         'apellido1',
18         'apellido2',
19         'genero_id',
20         'departamento_id',
21         'municipio_id',
22         'correo'
23     ];
24
25     public function tipoDocumento()
26     {
27         return $this->belongsTo(TipoDocumento::class);
28     }
29
30     public function genero()
31     {
32         return $this->belongsTo(Genero::class);
33     }
34
35     public function departamento()
36     {
37         return $this->belongsTo(Departamento::class);
38     }
39
40     public function municipio()
41     {
42         return $this->belongsTo(Municipio::class);
43     }
44 }
45
```

Tendríamos nuestro modelo al cual le generamos cada una de las relaciones y aparte le mostramos nuestra tabla principal.

Agregamos nuestras rutas de API y también modificamos cada controlador para ejecutar cada acción respecto al CRUD.


```

<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\{
    DepartamentoController,
    MunicipioController,
    TipoDocumentoController,
    GeneroController,
    PacienteController,
    UserController
};

Route::get('/prueba', function () {
    return response()->json(['status' => 'API funcionando correctamente']);
});

Route::apiResource('departamentos', DepartamentoController::class);
Route::apiResource('municipios', MunicipioController::class);
Route::apiResource('tipos-documento', TipoDocumentoController::class);
Route::apiResource('generos', GeneroController::class);
Route::apiResource('pacientes', PacienteController::class);
Route::apiResource('usuarios', UserController::class);

Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    return $request->user();
});

```

<pre> 11 use 12 use App\Http\Controllers 13 use App\Http\Controllers\ 14 use App\Http\Controllers\ 15 use App\Http\Controllers\ 16 use App\Http\Controllers\ 17 use App\Http\Controllers\ 18 use App\Http\Controllers\ 19 use App\Http\Controllers\ 20 use App\Http\Controllers\ 21 use App\Http\Controllers\ 22 use App\Http\Controllers\ 23 use App\Http\Controllers\ 24 use App\Http\Controllers\ 25 use App\Http\Controllers\ 26 use App\Http\Controllers\ 27 use App\Http\Controllers\ 28 use App\Http\Controllers\ 29 use App\Http\Controllers\ 30 use App\Http\Controllers\ 31 use App\Http\Controllers\ 32 use App\Http\Controllers\ 33 use App\Http\Controllers\ 34 use App\Http\Controllers\ 35 use App\Http\Controllers\ 36 use App\Http\Controllers\ 37 use App\Http\Controllers\ 38 use App\Http\Controllers\ 39 use App\Http\Controllers\ 40 use App\Http\Controllers\ 41 use App\Http\Controllers\ 42 use App\Http\Controllers\ 43 use App\Http\Controllers\ 44 use App\Http\Controllers\ 45 use App\Http\Controllers\ 46 use App\Http\Controllers\ 47 use App\Http\Controllers\ 48 use App\Http\Controllers\ 49 use App\Http\Controllers\ 50 use App\Http\Controllers\ 51 use App\Http\Controllers\ 52 use App\Http\Controllers\ 53 use App\Http\Controllers\ 54 use App\Http\Controllers\ 55 use App\Http\Controllers\ 56 use App\Http\Controllers\ 57 use App\Http\Controllers\ 58 use App\Http\Controllers\ 59 use App\Http\Controllers\ 60 use App\Http\Controllers\ 61 use App\Http\Controllers\ 62 use App\Http\Controllers\ 63 use App\Http\Controllers\ 64 use App\Http\Controllers\ 65 use App\Http\Controllers\ 66 use App\Http\Controllers\ 67 use App\Http\Controllers\ 68 use App\Http\Controllers\ 69 use App\Http\Controllers\ 70 use App\Http\Controllers\ 71 use App\Http\Controllers\ 72 use App\Http\Controllers\ 73 use App\Http\Controllers\ 74 use App\Http\Controllers\ 75 use App\Http\Controllers\ 76 use App\Http\Controllers\ 77 use App\Http\Controllers\ 78 use App\Http\Controllers\ 79 use App\Http\Controllers\ 80 use App\Http\Controllers\ 81 use App\Http\Controllers\ 82 use App\Http\Controllers\ 83 use App\Http\Controllers\ 84 use App\Http\Controllers\ 85 use App\Http\Controllers\ 86 use App\Http\Controllers\ 87 use App\Http\Controllers\ 88 use App\Http\Controllers\ 89 use App\Http\Controllers\ 90 use App\Http\Controllers\ 91 use App\Http\Controllers\ 92 use App\Http\Controllers\ 93 use App\Http\Controllers\ 94 use App\Http\Controllers\ 95 use App\Http\Controllers\ 96 use App\Http\Controllers\ 97 use App\Http\Controllers\ 98 use App\Http\Controllers\ 99 use App\Http\Controllers\ 100 use App\Http\Controllers\ </pre>	<pre> 11 use App\Http\Controllers\PacienteController as PacienteController; 12 public function index() 13 { 14 return response()->json(Paciente::with(['departamentos', 'municipios', 'generos', 'tiposDocumentos'])->get(), 200); 15 } 16 17 public function show(\$id) 18 { 19 \$paciente = Paciente::with(['departamentos', 'municipios', 'generos', 'tiposDocumentos'])->find(\$id); 20 if (!\$paciente) { 21 return response()->json(['error' => 'Paciente no encontrado'], 404); 22 } 23 return response()->json(\$paciente, 200); 24 } 25 26 public function store(Request \$request) 27 { 28 \$validated = \$request->validate([29 'tipo_documento_id' => 'required exists:tipos_documento,id', 30 'numero_documento' => 'required numeric unique:paciente,numero_documento', 31 'nombre' => 'required string max:100', 32 'apellido' => 'required string max:100', 33 'apellidos' => 'required string max:100', 34 'apellidos' => 'required string max:100', 35 'genero_id' => 'required exists:generos,id', 36 'departamento_id' => 'required exists:departamentos,id', 37 'municipio_id' => 'required exists:municipios,id', 38 'correo' => 'required email', 39]); 40 \$paciente = Paciente::create(\$validated); 41 return response()->json(\$paciente, 201); 42 } 43 44 public function update(Request \$request, \$id) 45 { 46 \$paciente = Paciente::find(\$id); 47 if (!\$paciente) { 48 return response()->json(['error' => 'Paciente no encontrado'], 404); 49 } 50 \$validated = \$request->validate([51 'numero_documento' => 'sometimes numeric unique:paciente,numero_documento,' . \$id, 52 'nombre' => 'sometimes string max:100', 53]); 54 \$paciente->update(\$validated); 55 return response()->json(\$paciente, 200); 56 } </pre>
--	---

De acuerdo a esto creamos nuestra colección en postman

Cada controlador tiene la respectiva validación de datos.

```

public function store(Request $request)
{
    $validated = $request->validate([
        'tipo_documento_id' => 'required|exists:tipos_documento,id',
        'numero_documento' => 'required|numeric|unique:paciente,numero_documento',
        'nombre1' => 'required|string|max:100',
        'nombre2' => 'nullable|string|max:100',
        'apellido1' => 'required|string|max:100',
        'apellido2' => 'nullable|string|max:100',
        'genero_id' => 'required|exists:genero,id',
        'departamento_id' => 'required|exists:departamentos,id',
        'municipio_id' => 'required|exists:municipios,id',
        'correo' => 'required|email',
    ]);

    $paciente = Paciente::create($validated);
    return response()->json($paciente, 201);
}

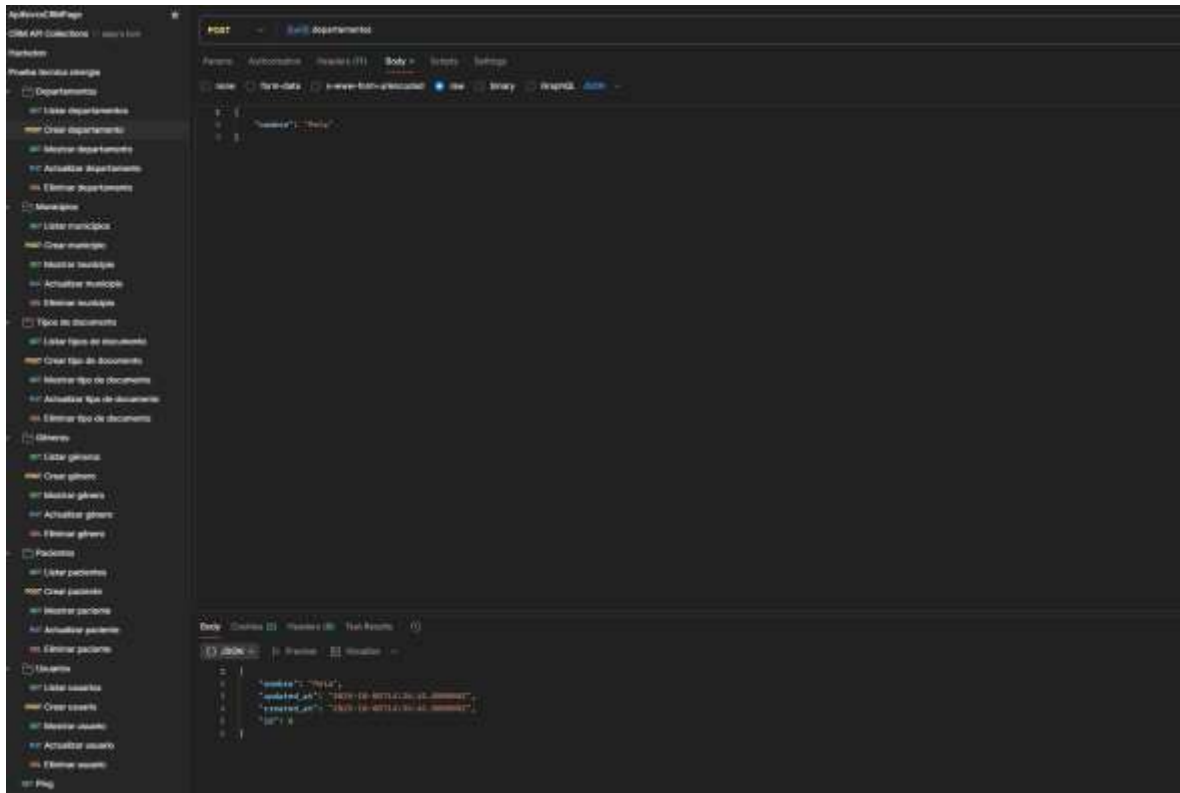
public function update(Request $request, $id)
{
    $paciente = Paciente::find($id);
    if (!$paciente) {
        return response()->json(['error' => 'Paciente no encontrado'], 404);
    }

    $validated = $request->validate([
        'numero_documento' => 'sometimes|numeric|unique:paciente,numero_documento,' . $id,
        'correo' => 'sometimes|email',
    ]);

    $paciente->update($validated);
    return response()->json($paciente, 200);
}

```

Creamos cada método y entrada con variables directas en postman



De esta manera logramos ya dejar una documentación de nuestra API y a su vez la forma en que nos debemos conectar a ella, por medio de una herramienta sencilla como lo es POSTMAN.

Para que nuestra API tenga seguridad implementaremos un componente de composer o un paquete que genera jwt, el cual se llama tymon para poder generar tokens y accesos.

```
> @php artisan vendor:publish --tag=laravel-assets --ansi --force
INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version ^2.2 for tymon/jwt-auth
root@907f7a2d6025:/var/www/html/laravel# php artisan vendor:publish --provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"

INFO Publishing assets.

Copying file [vendor/tymon/jwt-auth/config/config.php] to [config/jwt.php] .....
root@907f7a2d6025:/var/www/html/laravel#
root@907f7a2d6025:/var/www/html/laravel# php artisan jwt:secret
```

Generamos el secreto y el config de jwt.


```

use Illuminate\Support\Facades\Hash;
use Tymon\JWTAuth\Facades\JWTAuth;

class AuthController extends Controller
{
    public function register(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => 'required|string|min:8',
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        $token = JWTAuth::fromUser($user);

        return response()->json(['user' => $user, 'token' => $token], 201);
    }

    public function login(Request $request)
    {
        $credentials = $request->only('email', 'password');

        if (!$token = JWTAuth::attempt($credentials)) {
            return response()->json(['error' => 'Credenciales inválidas'], 401);
        }

        return response()->json(['token' => $token]);
    }

    public function me()
    {
        return response()->json(JWTAuth::parseToken()->authenticate());
    }

    public function logout()
    {
        JWTAuth::invalidate(JWTAuth::getToken());
        return response()->json(['message' => 'Sesión cerrada correctamente']);
    }
}

```

Agregamos la validación de sesión y autenticación.

```

8     TipoDocumentoController,
9     GeneroController,
10    PacienteController,
11    UserController,
12    AuthController
13 };
14
15
16
17 Route::get('/prueba', function () {
18     return response()->json(['status' => 'API funcionando correctamente']);
19 });
20
21 Route::apiResource('departamentos', DepartamentoController::class);
22 Route::apiResource('municipios', MunicipioController::class);
23 Route::apiResource('tipos-documento', TipoDocumentoController::class);
24 Route::apiResource('generos', GeneroController::class);
25 Route::apiResource('pacientes', PacienteController::class);
26 Route::apiResource('usuarios', UserController::class);
27
28 Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
29     return $request->user();
30 });
31
32 Route::post('register', [AuthController::class, 'register']);
33 Route::post('login', [AuthController::class, 'login']);
34
35 Route::middleware('auth:api')->group(function () {
36     Route::get('me', [AuthController::class, 'me']);
37     Route::post('logout', [AuthController::class, 'logout']);
38 });
39

```

Aquí agregamos los métodos de autenticación.

Agregamos la seguridad a todos nuestros métodos , dentro de un middleware que colocamos en nuestra api

```
3 use Illuminate\Http\Request;
4 use Illuminate\Support\Facades\Route;
5 use App\Http\Controllers\{
6     DepartamentoController,
7     MunicipioController,
8     TipoDocumentoController,
9     GeneroController,
10    PacienteController,
11    UserController,
12    AuthController
13 };
14
15
16
17 Route::get('/prueba', function () {
18     return response()->json(['status' => 'API funcionando correctamente']);
19 });
20
21
22 Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
23     return $request->user();
24 });
25
26 Route::post('register', [AuthController::class, 'register']);
27 Route::post('login', [AuthController::class, 'login']);
28
29 Route::middleware(['auth:api'])->group(function () {
30
31     // Usuario autenticado
32     Route::get('/me', [AuthController::class, 'me']);
33     Route::post('/logout', [AuthController::class, 'logout']);
34
35     // CRUDs protegidos
36     Route::apiResource('departamentos', DepartamentoController::class);
37     Route::apiResource('municipios', MunicipioController::class);
38     Route::apiResource('tipos-documento', TipoDocumentoController::class);
39     Route::apiResource('generos', GeneroController::class);
40     Route::apiResource('pacientes', PacienteController::class);
41     Route::apiResource('usuarios', UserController::class);
42 }, );
```

Agregamos otro middleware para que nuestras peticiones si falla el token no nos arroje error 500 sino que nos pueda dar un mensaje de status

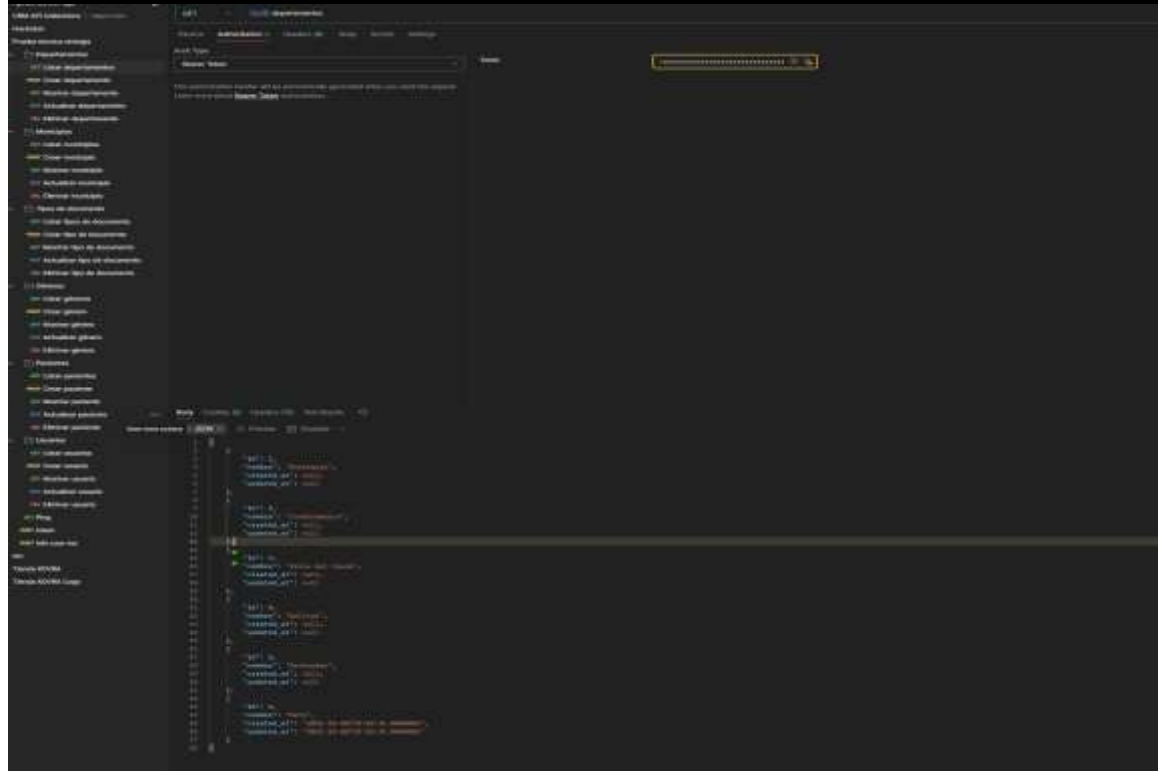
```
<?php

namespace App\Http\Middleware;

use Closure;
use Exception;
use Tymon\JWTAuth\Facades\JWTAuth;
use Tymon\JWTAuth\Exceptions\TokenExpiredException;
use Tymon\JWTAuth\Exceptions\TokenInvalidException;
use Tymon\JWTAuth\Exceptions\JWTException;

class JwtMiddleware
{
    public function handle($request, Closure $next)
    {
        try {
            // Verifica el token y autentica el usuario
            JWTAuth::parseToken()->authenticate();
        } catch (TokenExpiredException $e) {
            return response()->json(['error' => 'El token ha expirado'], 401);
        } catch (TokenInvalidException $e) {
            return response()->json(['error' => 'Token inválido'], 401);
        } catch (JWTException $e) {
            return response()->json(['error' => 'Token no proporcionado'], 401);
        } catch (Exception $e) {
            return response()->json(['error' => 'Error en la autenticación'], 500);
        }

        return $next($request);
    }
}
```



Generación de pruebas con PHPUNIT

```
root@907f7a2d6025:/var/www/html/laravel# php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response

Tests: 2 passed (2 assertions)
Duration: 12.35s
```

De igual manera crearemos un test para validar la autenticación.

```
AuthTest.php
1 <?php
2
3 namespace Tests\Feature;
4
5 use Illuminate\Foundation\Testing\RefreshDatabase;
6 use Tests\TestCase;
7 use App\Models\User;
8
9 class AuthTest extends TestCase
10 {
11     use RefreshDatabase;
12
13
14     public function usuario_puede_registrarse()
15     {
16         $response = $this->postJson('/api/register', [
17             'name' => 'Carlos',
18             'email' => 'carlos@ingedeu94.com',
19             'password' => '1234567890',
20         ]);
21
22         $response->assertStatus(201)
23         | | |>assertJsonStructure(['user', 'token']);
24     }
25
26     public function usuario_puede_iniciar_sesion()
27     {
28         $user = User::factory()->create([
29             'password' => bcrypt('1234567890')
30         ]);
31
32         $response = $this->postJson('/api/login', [
33             'email' => $user->email,
34             'password' => '1234567890',
35         ]);
36
37         $response->assertStatus(200)
38         | | |>assertJsonStructure(['access_token']);
39     }
40
41     public function no_autenticado_no_puede_acceder_a_rutas_protegidas()
42     {
43         $response = $this->getJson('/api/pacientes');
44
45         $response->assertStatus(401)
46         | | |>assertJson(['error' => 'Token no proporcionado']);
47     }
48 }
49
```

Creamos dos uno para garantizar los accesos y registros y el otro para ver los pacientes y hacer pruebas con el endpoint de pacientes.

```
namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;
use App\Models\User;
use App\Models\Paciente;

class PacienteTest extends TestCase
{
    use RefreshDatabase;

    protected $token;

    protected function setUp(): void
    {
        parent::setUp();

        $user = User::factory()->create([
            'password' => bcrypt('1234567890')
        ]);

        $response = $this->postJson('/api/login', [
            'email' => $user->email,
            'password' => '1234567890',
        ]);

        $this->token = $response->json('access_token');
    }

    public function puede_listar_pacientes()
    {
        $response = $this->withHeader('Authorization', "Bearer {$this->token}")
            ->getJson('/api/pacientes');

        $response->assertStatus(200);
    }

    public function puede_crear_un_paciente()
    {
        $data = [
            'tipo_documento_id' => 1,
            'numero_documento' => 123456,
            'nombre' => 'Juan',
            'apellidos' => 'Pérez',
            'genero_id' => 1,
            'departamento_id' => 1,
            'municipio_id' => 1,
            'correo' => 'juan@example.com'
        ];
    }
}
```

Verificamos el test con errores

```

C:\Program Files\MySQL\MySQL Server 5.6>mysql -u root -p
mysql> use test;
mysql> create table test (id int(11) unsigned not null auto-increment primary key, 'created at' timestamp(14) not null);
mysql> insert into test (id, 'created at') values (1, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (2, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (3, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (4, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (5, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (6, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (7, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (8, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (9, '2016-01-01 00:00:00.000000');
mysql> insert into test (id, 'created at') values (10, '2016-01-01 00:00:00.000000');
mysql> select * from test;
+----+-----+
| id | 'created at' |
+----+-----+
|  1 | 2016-01-01 00:00:00.000000 |
|  2 | 2016-01-01 00:00:00.000000 |
|  3 | 2016-01-01 00:00:00.000000 |
|  4 | 2016-01-01 00:00:00.000000 |
|  5 | 2016-01-01 00:00:00.000000 |
|  6 | 2016-01-01 00:00:00.000000 |
|  7 | 2016-01-01 00:00:00.000000 |
|  8 | 2016-01-01 00:00:00.000000 |
|  9 | 2016-01-01 00:00:00.000000 |
| 10 | 2016-01-01 00:00:00.000000 |
+----+-----+
mysql>

```



```

use RefreshDatabase;

/** @test */
public function usuario_puede_registrarse()
{
    $response = $this->postJson('/api/register', [
        'name' => 'Carlos',
        'email' => 'carlos@example.com',
        'password' => '1234567890',
    ]);

    $response->assertStatus(201)
        |>assertJsonStructure(['user', 'token']);
}

/** @test */
public function usuario_puede_iniciar_sesion()
{
    $user = User::factory()->create([
        'password' => bcrypt('1234567890')
    ]);

    $response = $this->postJson('/api/login', [
        'email' => $user->email,
        'password' => '1234567890',
    ]);

    $response->assertStatus(200)
        |>assertJsonStructure(['access_token']);
}

/** @test */
public function no_autenticado_no_puede_acceder_a_rutas_protegidas()
{
    $response = $this->getJson('/api/pacientes');

    $response->assertStatus(401)

```

Este error sucede por que esta buscando una base de pruebas, pero para esta ocasión vamos a ejecutar la prueba en nuestra misma base prod por lo que cambiamos nuestra refreshdatabase por withouthmiddleware para que se ejecute allí.

```
FAIL Tests\Feature\AuthTest
✓ usuario puede registrarse
✗ usuario puede iniciar sesion
✗ no autenticado no puede acceder a rutas protegidas

FAILED Tests\Feature\AuthTest > usuario puede iniciar sesion
Failed asserting that an array has the key 'access_token'.

at tests/Feature/AuthTest.php:36
32         'password' => '1234567890',
33     ];
34
35     $response->assertStatus(200)
→ 36         ->assertJsonStructure(['access_token']);
37 }
38
39 /** @test */
40 public function no_autenticado_no_puede_acceder_a_rutas_protegidas()

FAILED Tests\Feature\AuthTest > no autenticado no puede acceder a rutas protegidas
Expected response status code [401] but received 200.
Failed asserting that 200 is identical to 401.

at tests/Feature/AuthTest.php:44
40 public function no_autenticado_no_puede_acceder_a_rutas_protegidas()
41 {
42     $response = $this->getJson('/api/pacientes');
43
→ 44     $response->assertStatus(401)
45         ->assertJson(['error' => 'Token no proporcionado']);
46 }
47 }
48

Tests: 2 failed, 1 passed (6 assertions)
Duration: 12.86s
```

Perfecto ahora ya tenemos una prueba correcta que es el de registro y dos que son de acceso invalidas.

Este error se debe por que la respues la recibe no como Access_token sino como token.


```

root@987177azd6025:/var/www/html/laravel# php artisan test --filter=AuthTest

WARN Metadata found in doc-comment for method Tests\Feature\AuthTest

WARN Metadata found in doc-comment for method Tests\Feature\AuthTest

WARN Metadata found in doc-comment for method Tests\Feature\AuthTest

PASS Tests\Feature\AuthTest
✓ usuario puede registrarse
✓ usuario puede iniciar sesion
✓ no autenticado no puede acceder a rutas protegidas

Tests: 3 passed (7 assertions)
Duration: 13.17s

```

Ahora si la prueba es correcta y se ejecuto correctamente, de esta manera realizamos pruebas con unit a nuestra plataforma de pacientes.

FRONT

Ahora para nuestro front para no generar otra codificación vamos a generarla dentro de nuestro proyecto laravel con blade para generar el contenido visual, de igual manera como ya tenemos una API podríamos conectarnos a ella por medio de node con angular, react, flutter, ionic o demás lenguajes de front para generar la visual pero consumiendo una API segura y confiable como la que generamos.

Para ello usaremos una arquitectura de capas donde pondremos un dashboard y este tendrá un archivo que controlara el menú nuestro header, otro que será la visual que es nuestro archivo app y también nuestro footer y dentro de ello cada vista interna será independiente para poder visualizar cada modulo.


```

el > resources > views > inicio.blade.php > div.container.py-4 > div.row.mt-5 > div.col-md-4.mb-4 > div.card.shadow-sm.border-0.h-1
@extends('base.app')

@section('title', 'Inicio')

@section('content')
<div class="container py-4">
  <div class="text-center">
    <h2 class="fw-bold mb-3">Bienvenido al Panel de Control</h2>
    <p>Selecciona una opción del menú para gestionar los módulos del sistema.</p>
  </div>

  <div class="row mt-5">
    <div class="col-md-4 mb-4">
      <div class="card shadow-sm border-0 h-100">
        <div class="card-body text-center">
          <i class="bi bi-person-vcard display-5 text-primary mb-3"></i>
          <h5>Pacientes</h5>
          <p>Administra los registros de pacientes.</p>
          <a href="/pacientes" class="btn btn-outline-primary btn-sm">Ver módulo</a>
        </div>
      </div>
    </div>

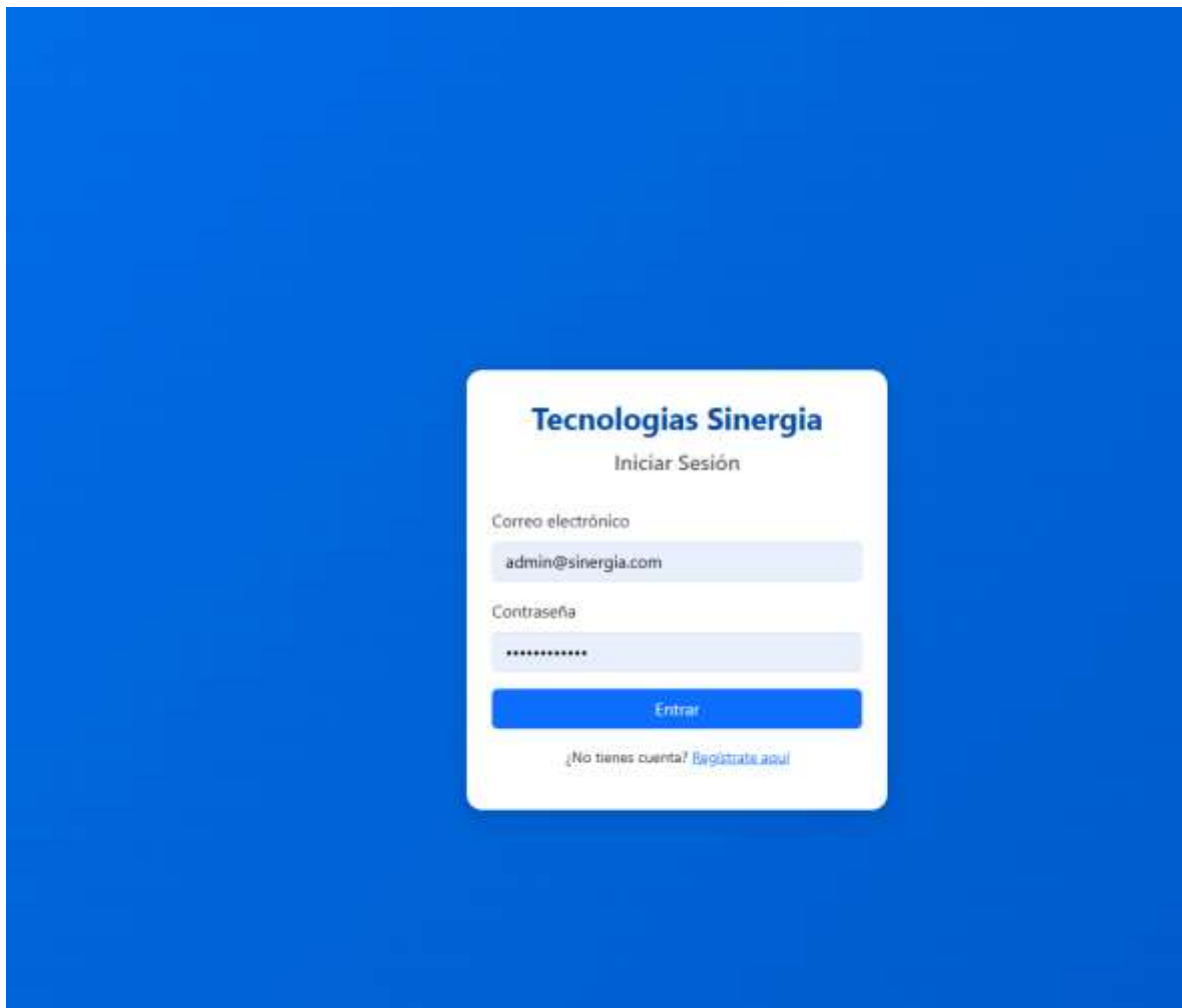
    <div class="col-md-4 mb-4">
      <div class="card shadow-sm border-0 h-100">
        <div class="card-body text-center">
          <i class="bi bi-people display-5 text-primary mb-3"></i>
          <h5>Usuarios</h5>
          <p>Gestión de cuentas del sistema.</p>
          <a href="#" class="btn btn-outline-primary btn-sm">Ver módulo</a>
        </div>
      </div>
    </div>

    <div class="col-md-4 mb-4">
      <div class="card shadow-sm border-0 h-100">
        <div class="card-body text-center">
          <i class="bi bi-gear display-5 text-primary mb-3"></i>
          <h5>Configuración</h5>
          <p>Administra parámetros del sistema.</p>
          <a href="#" class="btn btn-outline-primary btn-sm">Ver módulo</a>
        </div>
      </div>
    </div>
  </div>
</div>
@endsection

```

Extendemos de nuestro archivo app principal pero mostrando otro contenido.

De acuerdo a que ya tenemos las vista y los modulos podemos generar un login con middleware para que valida la sesión del usuario y pueda ingresar a la herramienta, de acuerdo a ello, a continuación te muestro las rutas como están modificadas, y también el registro de usuario y login



```
const logoutBtn = document.getElementById('btnLogout');
if (logoutBtn) {
  logoutBtn.addEventListener('click', () => {
    if (confirm('¿Seguro que deseas cerrar sesión?')) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
  });
}
```

```

<?php

use Illuminate\Support\Facades\Route;

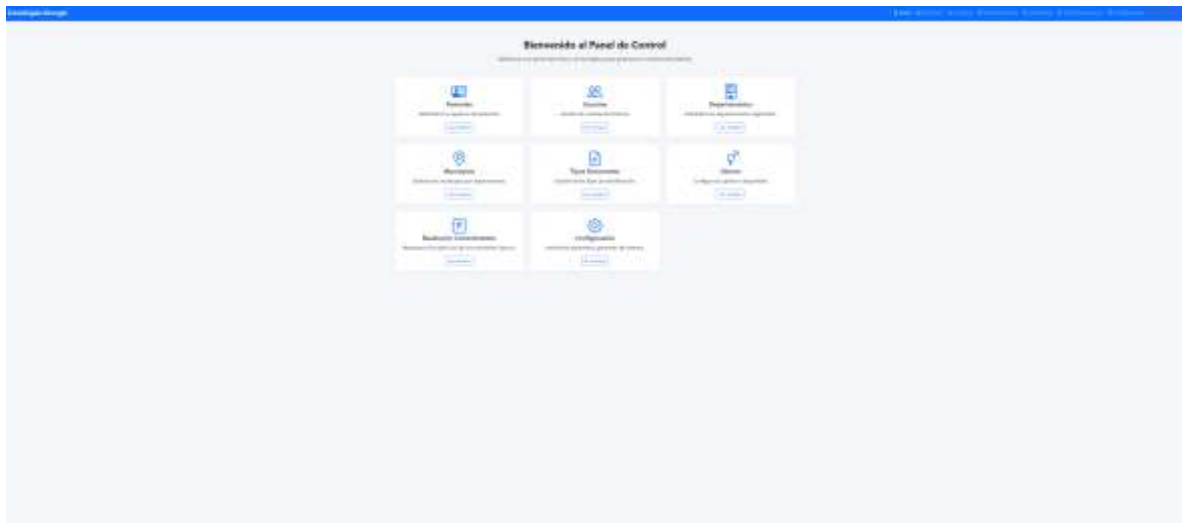
Route::view('/login', 'auth.login')->name('login');
Route::view('/register', 'auth.register')->name('register');

Route::get('/', fn() => view('inicio'))->name('dashboard');
Route::get('/pacientes', fn() => view('pacientes'))->name('pacientes');
Route::get('/usuarios', fn() => view('usuarios'))->name('usuarios');
Route::get('/departamentos', fn() => view('departamentos'))->name('departamentos');
Route::get('/municipios', fn() => view('municipios'))->name('municipios');
Route::get('/tipos-documento', fn() => view('tipos-documento'))->name('tipos-documento');
Route::get('/genero', fn() => view('genero'))->name('genero');
Route::get('/configuracion', fn() => view('configuracion'))->name('configuracion');
Route::get('/resolucion-conocimientos', fn() => view('resolucion-conocimientos'))->name('resolucion-conocimientos');

Route::fallback(function () {
    return redirect('/login');
});

```

Aca estamos realizando la inscripcion de las rutas.



Aquí ya tendríamos el funcionamiento de nuestro dashboard con nuestros modulos

Pacientes + Nuevo

ID	Nombre completo	Correo	Tipo Doc	N° Doc	Sexo	Municipio	Acciones
1	Carlos Maldonado	carlos@example.com	1	10000001	M	1	☒ ☐
2	Luisa Garcia	luisa@example.com	2	10000002	F	2	☒ ☐
3	Miguel Torres	miguel@example.com	1	10000003	M	3	☒ ☐
4	Valentina Castro	valentina@example.com	1	10000004	F	1	☒ ☐
5	Andrés Sánchez	andres@example.com	2	10000005	M	5	☒ ☐

De igual manera agregamos validaciones a los formularios

```
function validarFormulario(){
  let valido=true;
  const correo=document.getElementById('correo');
  const doc=document.getElementById('numero_documento');
  document.querySelectorAll('#formPaciente input[required]').forEach(i=>{
    if(!i.value.trim()){i.classList.add('is-invalid');valido=false;}
    else i.classList.remove('is-invalid');
  });
  if(!/^\d{5,20}$/.test(doc.value)){doc.classList.add('is-invalid');valido=false;}
  if(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(correo.value)){correo.classList.add('is-invalid');valido=false;}
  return valido;
}
```

The image shows a web form titled "Nuevo Paciente" (New Patient) with a blue header and a white body. The form contains several input fields, each with a red border and a red error message below it. The fields and their errors are:

- Tipo Documento:** A dropdown menu with a red error message: "Ingrese un tipo de documento válido."
- Número Documento:** A text input field with a red error message: "Ingrese un número de documento válido (solo números, mínimo 5 dígitos)."
- Primer Nombre:** A text input field with a red error message: "El nombre solo debe contener letras."
- Primer Apellido:** A text input field with a red error message: "El apellido solo debe contener letras."
- Género:** A dropdown menu with a red error message: "Seleccione un género válido."
- Municipio ID:** A text input field with a red error message: "Ingrese un municipio válido."
- Correo:** A text input field with a red error message: "Ingrese un correo válido."

At the bottom right of the form, there are two buttons: "Cancelar" (Cancel) and "Guardar" (Save).

ANEXO POSTMAN COMPLETO DE API PARA PODER CONECTARSE Y EJECUTARLA DE IGUAL MANERA EN EL REPO SE ENCUENTRA EL DOCKER PARA PODER INTALAR EL AMBIENTE Y EJECUTARLO POR MEDIO DE UN CONTENEDOR, ALLI ENCONTRARAS EL JSON DEL POSTMAN EL CUAL PODRAS COMPROBAR CON TODOS LOS METODOS Y SU VARIABLE DE URL.

✓	Prueba tecnica sinergia
✓	Departamentos
	GET Listar departamentos
	POST Crear departamento
	GET Mostrar departamento
	PUT Actualizar departamento
	DEL Eliminar departamento
✓	Municipios
	GET Listar municipios
	POST Crear municipio
	GET Mostrar municipio
	PUT Actualizar municipio
	DEL Eliminar municipio
✓	Tipos de documento
	GET Listar tipos de documento
	POST Crear tipo de documento
	GET Mostrar tipo de documento
	PUT Actualizar tipo de documento
	DEL Eliminar tipo de documento
✓	Géneros
	GET Listar géneros
	POST Crear género
	GET Mostrar género
	PUT Actualizar género
	DEL Eliminar género
✓	Pacientes
	GET Listar pacientes
	POST Crear paciente
	GET Mostrar paciente
	PUT Actualizar paciente
	DEL Eliminar paciente
✓	Usuarios
	GET Listar usuarios
	POST Crear usuario
	GET Mostrar usuario
	PUT Actualizar usuario
	DEL Eliminar usuario
	GET Ping
	POST token
	POST info user me